

OOPS Concept

Keywords in Java :-

- Keywords in Java are reserved words that have special meaning to the compiler.
- They cannot be used as identifiers such as variable names, class names, or method names.
- Java keywords define the structure, flow, and functionality of a program.
- They are all written in lowercase and are predefined by the Java language.
- There are 67 keywords in Java (as per the latest version).

boolean	byte	char	double	float
short	void	int	long	while
for	do	switch	break	continue
case	default	if	else	try
catch	finally	class	abstract	extends
final	import	new	instance of	private
interface	native	public	package	implements
protected	return	static	super	synchronized
this	throw	throws	transient	volatile

1. Access Modifiers -

- 1) **public** - Accessible everywhere

Ex. public class A {}

- 2) **private** - Accessible only in same class

Ex. private int x;

- 3) **protected** - Accessible in package & subclass

Ex. protected void show(){}

- 4) **default** - Package-private (no keyword)

Ex. int a=10

2. Class & Object Related -

- 1) **class** - Defines class

Ex. class Student {}

- 2) **interface** - Defines interface

Ex. interface A { void show(); }

- 3) **extends** - Inherit a class

Ex. class B extends A{}

- 4) **implements** - Implement interface

Ex. class C implements A{}

- 5) **new** - Create object

Ex. Student s = new Student();

- 6) **this** - Refers to current object

Example 1 –

```
package oops;

public class person1 {
    String name;

    void info(String name)
    {
        this.name=name;
        System.out.println(name);
    }

    public static void main(String[] args) {
        person1 p1=new person1();
        p1.info("Sagar");
        System.out.println(p1.name);
    }
}
```

Output –

Sagar

Sagar

Example 2 –

```
package oops;

public class person2 {
    void walk()
    {
        System.out.println("Walking");
    }

    void run()
    {
        this.walk();
        System.out.println("Running");
    }

    public static void main(String[] args) {
        person2 p2=new person2();
        p2.run();
    }
}
```

Output –

Walking

Running

7) **super** - Refers to parent class

Ex. super.display();

8) **abstract** - Abstract class/method

Ex. abstract class Shape {}

9) **final** - Constant / prevent inheritance

Ex. final int x=10;

10) **static** - Class-level member

Ex. static int count;

3. Data Types -

1) **int** - Integer

Ex. int a=5;

2) **byte** - 8-bit integer

Ex. byte b=10;

3) **short** - 16-bit integer

Ex. short s=20;

4) **long** - 64-bit integer

Ex. long l=1000L;

5) **float** - 32-bit decimal

Ex. float f=3.14f;

6) **double** - 64-bit decimal

Ex. double d=9.81;

7) **char** - Character

Ex. char c='A';

8) **boolean** - True/False

Ex. boolean b=true;

9) **void** - No return

Ex. void show(){}

4. Control Flow -

1) **if** - Condition

Ex. `if(a>5){}`

2) **else** - Else condition

Ex. `else{}`

3) **switch** - Multi-way branch

Ex. `switch(a){ case 1: break;}`

4) **case** - Case inside switch

Ex. `case 2: break;`

5) **for** - Loop

Ex. `for(int i=0;i<5;i++){}`

6) **while** - Loop

Ex. `while(i<5){}`

7) **do - Loop** executes once first

Ex. `do{} while(i<5);`

8) **break** - Exit loop/switch

Ex. `break;`

9) **continue** - Skip iteration

Ex. `continue;`

10) **return** - Exit method / return value

Ex. `return x;`

5. Exception Handling -

- 1) **try** - Test block

Ex. try{}

- 2) **catch** - Handle exception

Ex. catch(Exception e){}

- 3) **finally** - Always executes

Ex. finally{}

- 4) **throw** - Throw exception

Ex. throw new Exception();

- 5) **throws** - Declare exception

Ex. void m() throws IOException{}

6. Packages & Object -

- 1) **package** - Define package

Ex. package mypack;

- 2) **import** - Import class/package

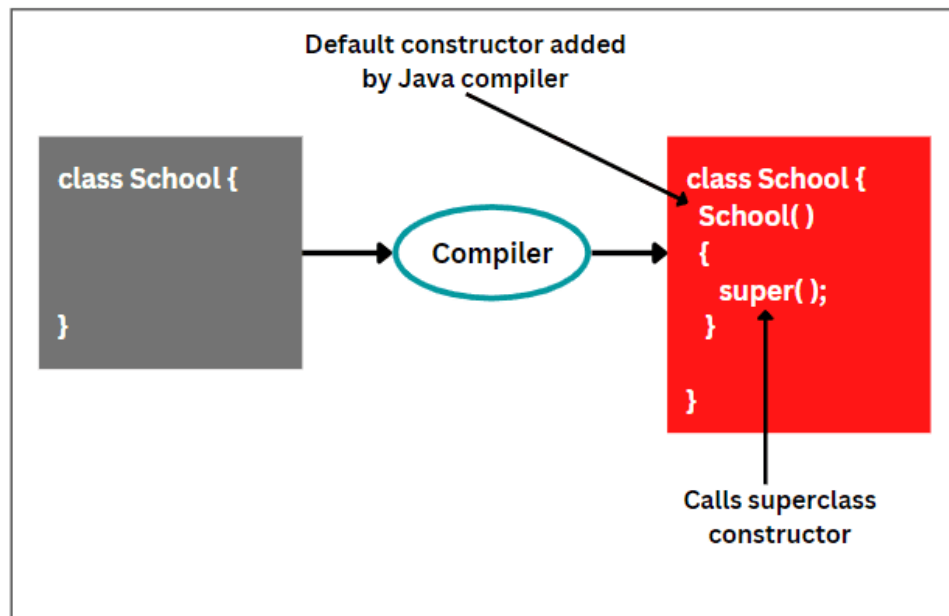
Ex. import java.util.*;

- 3) **instanceof** - Test object type

Ex. if(obj instanceof String){}

Constructor in Java :-

- A constructor in Java is a special method used to initialize objects.
- It has the same name as the class.
- It does not have a return type, not even void.
- It is automatically called when an object is created.
- Constructors can be overloaded (multiple constructors in one class).



Syntax of Constructor :-

```
class ClassName {  
  
    // Default constructor  
  
    ClassName() {  
  
        // body of constructor  
  
    }  
  
}
```

Rules of Constructor :-

1. The constructor name must be the same as the class name and it has no return type.
2. It is automatically called when an object is created.
3. A class can have multiple constructors (constructor overloading).
4. `this()` and `super()` can be used to call other constructors.
5. A constructor cannot be static, abstract, final, or synchronized.

When to use a Constructor :-

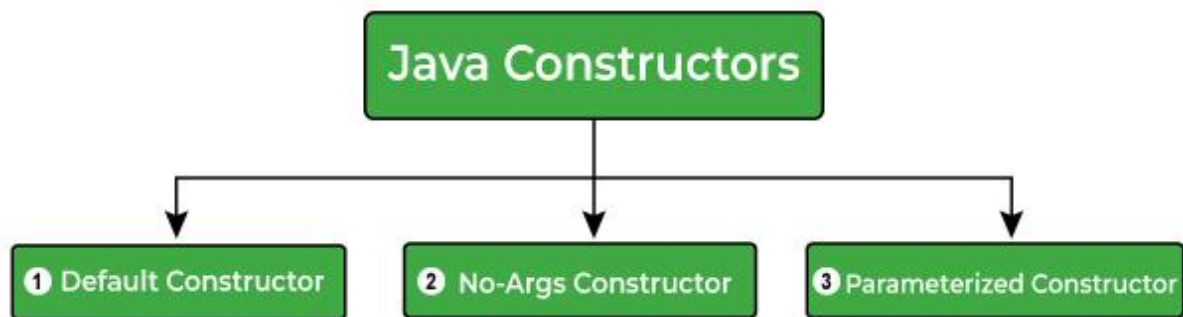
- When you want to initialize object variables at the time of object creation.
- When you want to assign default values to fields without writing extra methods.
- When you need different ways to initialize objects using constructor overloading.
- When you want to enforce that an object cannot exist without required values (via parameterized constructor).
- When you need to call parent class initialization using `super()`.

Difference Between Constructor & Method :-

Features	Constructor	Method
Name	Constructors must have the same name as the class name	Methods can have any valid name
Return Type	Constructors do not return any type	Methods have the return type or void if does not return any value.
Invocation	Constructors are called automatically with new keyword	Methods are called explicitly
Purpose	Constructors are used to initialize objects	Methods are used to perform operations

Types of Constructor :-

- A constructor in Java is a special method used to initialize objects.
- Based on how they initialize objects, constructors are divided into different types.
- The main types of constructors are:



1. Default Constructor :-

- A Default Constructor is a constructor that does not take any parameters.
- It is used to assign default values to the object's variables.
- If no constructor is defined in a class, Java provides a default constructor automatically.
- It is automatically called when an object is created.

Syntax :-

```
class ClassName {  
  
    ClassName() {  
  
        // constructor body  
  
    }  
  
}
```

Example –

```
package constructor;  
  
public class default_constructor {  
    default_constructor() //creating the constructor  
    {  
        System.out.println("Hello default constructor");  
    }  
  
    default_constructor(int a) //parameterized constructor  
    {  
        System.out.println("Area of circle is: "+3.14*a*a);  
    }  
  
    public static void main(String[] args) {  
        //creation of object(calling default constructor)  
        default_constructor obj1=new default_constructor();  
        default_constructor obj2=new default_constructor(4);  
    }  
}
```

Output –

```
Hello default constructor  
Area of circle is: 50.24
```